

Vector

O vector é uma estrutura de dados que representa um array dinâmico, ou seja, um array cujo tamanho pode aumentar e diminuir durante a execução do programa. Além disso ele disponibiliza vários métodos para manipulação dos seus elementos.

É semelhante a um array que vemos em FUP: seus elementos possuem um mesmo tipo e são identificados por um índice que vai de 0 a n-1, mas permite que o tamanho seja alterado dinamicamente.

Declaração

Para declarar um vetor vazio, com zero elementos:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    // vetor vazio
    vector<int> V;
}
```

Para declarar um vetor com um tamanho inicial e valor padrão:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    // vetor com 10 elementos, iguais a 0
    vector<int> V(10);
    // vetor com 10 elementos, iguais a 2
    vector<int> V(10, 2);
}
```

Inserção

Você pode adicionar um elemento `x` no final do vetor usando o método `push_back(x)`:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    // Vetor vazio
    vector<int> V;
    V.push_back(8);
    V.push_back(5);
}
```

```
V.push_back(10);  
}
```

Após a execução do código acima, o vetor possui os elementos [8, 5, 10]. Você pode usar o método `size` para obter o tamanho do vetor e imprimí-lo:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main(){  
    // Vetor vazio  
    vector<int> V;  
    V.push_back(8);  
    V.push_back(5);  
    V.push_back(10);  
    for(int i = 0; i < V.size(); i++){ // percorre os indices do vetor  
        cout << V[i] << endl;  
    }  
}
```

Você pode alterar o tamanho do vetor para N (para menor ou maior) utilizando o método `resize(N)`. Caso o tamanho aumente, ele irá preencher os novos espaços com o elemento nulo, sendo 0 para o tipo inteiro:

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main(){  
    // Vetor vazio  
    vector<int> V;  
    V.push_back(8);  
    V.push_back(5);  
    V.push_back(10);  
    // Temos V = [8,5,10]  
    V.resize(5);  
    // Temos V = [8, 5, 10, 0, 0]  
}
```

Acesso

O acesso aos elementos do vetor é feito da mesma forma que um array tradicional, utilizamos os colchetes e o índice (iniciando por 0) para acessar um elemento específico.

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main(){  
    vector<int> V(3);  
    // Temos V = [0, 0, 0]  
}
```

```
V[0] = 7;
V[1] = 3;
V[2] = 15;
// Temos V = [7, 3, 15]

cout << V[2] << endl;
// Imprime 15
}
```

Remoção

Podemos remover o último elemento do vetor utilizando o método `pop_back()`:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> V(3);
    // Temos V = [0, 0, 0]

    V[0] = 7;
    V[1] = 3;
    V[2] = 15;
    // Temos V = [7, 3, 15]

    V.pop_back();
    // Temos V = [7, 3]
    V.pop_back();
    // Temos V = [7]
}
```

Podemos também remover todos os elementos do vetor utilizando o método `clear()`:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> V(3);
    // Temos V = [0, 0, 0]

    V[0] = 7;
    V[1] = 3;
    V[2] = 15;
    // Temos V = [7, 3, 15]

    V.clear();
    // Temos V = []
}
```

É bem comum o uso do `clear` quando temos problemas que envolvem vários casos de teste.

Ordenção

Você pode ordenar os elementos do vetor utilizando a função `sort` da biblioteca padrão:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    vector<int> V;
    V.push_back(7);
    V.push_back(3);
    V.push_back(15);
    V.push_back(1);
    V.push_back(9);
    // Temos V = [7, 3, 15, 1, 9]

    sort(V.begin(), V.end());
    // Temos V = [1, 3, 7, 9, 15]
}
```

Os valores `V.begin()` e `V.end()` são ponteiros para o início e para o final da coleção de elementos.

Operações e Complexidade

A estrutura `vector` permite uma gama maior de operações, você pode consultar a documentação completa na [documentação](#). Mas tenha cuidado: nem toda operação é eficiente! Você estudará em ED que toda estrutura tem suas limitações.

Segue a complexidade de algumas das operações do `vector`, dado o tamanho do vetor n :

- `push_back(x)`: $O(1)$
- `push_front(x)`: $O(n)$
- `pop_back()`: $O(1)$
- `pop_front()`: $O(n)$
- `size()`: $O(1)$
- `operator[]`: $O(1)$
- `resize(m)`: $O(n + m)$
- `clear()`: $O(n)$